

TRINETRA

Member 3 — Vaultkeeper Backup & Recovery Engineer

A complete beginner-to-implementation guide

Mission	Protect clean recovery copies, maintain versions, verify integrity, and restore affected files automatically after a confirmed ransomware event.
Your main output	vaultkeeper/ → backup + versioning + integrity + restore + recovery reporting
Project	TRINETRA — Bharat's Next-Generation Cyber Resilience Platform
Core tagline	Detect → Defend → Auto-Recover

How to use this PDF: Read it from page 1 in order. You do not need to know ransomware, backups, hashes, APIs, Python architecture, or Remberall beforehand. The guide starts from zero and gradually moves toward the exact module you need to build.

Source basis: TRINETRA Detailed Idea Updated (20 pages) and the provided Smart India Hackathon 2026 deck (6 pages). The guide preserves the project's terminology and responsibilities; implementation details that are not explicitly specified in those sources are marked as suggested implementation choices.

1. Your Role in One Minute

You are Member 3: Recovery Engineer. Your module is called **Vaultkeeper**. The project assigns you ownership of Vaultkeeper + Remberall, with the main output **vaultkeeper/**. The team flow is: Member 1 detects activity → Member 2 analyzes it → Member 4 makes the policy decision → Member 6 handles containment → **you recover the data** → Member 5 visualizes the result. [TRINETRA, p. 17]

The official design describes Vaultkeeper as the component that maintains protected recovery copies and performs automatic recovery. It also specifies Remberall integration, AES-256 protection, versioning, integrity verification, selective restoration, and recovery reporting. [TRINETRA, pp. 4, 7–8]

Your job is not to detect ransomware. Your job begins when the system has a trusted recovery decision. You must answer four questions: (1) What files were affected? (2) Which clean version of each file should be used? (3) Can that version be trusted? (4) Can it be restored and verified safely?

What you will learn

- Backups, versions, snapshots, hashes, checksums, integrity, encryption and vaults.
- How Vaultkeeper fits into TRINETRA.
- How to design the vault and metadata.
- How SHA-256 integrity verification works.
- How to select a clean recovery version.
- How AES-256 fits into backup protection.
- How Remberall fits into the project's recovery layer.
- How Vaultkeeper communicates with FastAPI and the dashboard.
- How to build and test the module step by step.
- How to demonstrate it safely using a simulator.

3. Beginner Concepts

Term	Beginner explanation
Backup	A separate copy of data that can be used when the original is damaged or lost.
Recovery	Putting a clean copy back where the damaged file used to be.
Version	A saved state of a file at a particular point in time.
Snapshot	A recorded point-in-time state of files.
Hash	A fixed-length fingerprint calculated from file contents.
SHA-256	A cryptographic hashing algorithm used here for integrity checks.
Integrity	Confidence that the recovered file is the expected, uncorrupted file.
Vault	Protected storage containing recovery copies, versions and metadata.
Encryption	Transforming readable data into protected ciphertext.
AES-256	The backup-encryption method specified by the project.
Deduplication	Avoiding unnecessary duplicate storage.

Term	Beginner explanation
Selective recovery	Restoring only damaged files.

If these words are clear, most of the rest of your role becomes straightforward: **keep versions** → **identify a trusted version** → **verify it** → **restore it** → **verify again**.

4. The Most Important Rule: Never Restore Blindly

Suppose an attack begins at 12:05. You have versions from 10:00, 11:00, 12:00 and 12:06. The newest version might already contain encrypted data. Therefore, the newest backup is not automatically the safest backup.

```
10:00 -> V1 -> CLEAN
11:00 -> V2 -> CLEAN
12:00 -> V3 -> CLEAN
12:05 -> RANSOMWARE ATTACK
12:06 -> V4 -> POTENTIALLY COMPROMISED
```

```
Recovery:
find candidate before the trusted attack boundary
    ->
verify integrity
    ->
choose latest VERIFIED CLEAN version
    ->
restore
```

The TRINETRA specification explicitly calls for quick identification of clean versions, integrity verification and selective recovery. [TRINETRA, pp. 7–8]

5. Your Eight Responsibilities

Responsibility	What you actually do
Identify critical files/folders	Know which user data TRINETRA is protecting.
Create protected recovery copies	Create copies plus the metadata needed to validate them later.
Maintain versions/snapshots	Keep multiple safe states instead of overwriting the only backup.
Verify file integrity	Verify candidate backups and verify restored files.
Integrate Remberall	Use the project's recovery/versioning layer for snapshots, integrity, deduplication and recovery.
Restore affected files	Find clean versions and restore only damaged files.
Track recovered vs lost	Return per-file results and totals.
Generate recovery information	Provide data for the Business Recovery Report.

These responsibilities come directly from the Member 3/Vaultkeeper specification. [TRINETRA, pp. 7, 17]

6. Understand the Vault

The vault is a protected library of clean file states. It is separate from the user's normal working files.

```
User files
|
| backup
```

```

v
Vaultkeeper
|
+--> protected storage
|   +--> report.docx : v1
|   +--> report.docx : v2
|   +--> report.docx : v3
|
+--> metadata
    +--> timestamp
    +--> size
    +--> SHA-256
    +--> version ID
    +--> status

```

The project specifies protected recovery copies, versioning, integrity checks and AES-256 encrypted backup. The exact physical storage layout is not prescribed, so the diagram is a suggested implementation model. [TRINETRA, pp. 4, 7, 15]

7. Design Your Metadata First

Before writing recovery code, decide what you need to remember. A useful record contains:

- version_id
- original/source path
- vault reference/path
- timestamp
- file size
- SHA-256 hash/checksum
- version number
- trust/status
- recovery result

Field	Example	Purpose
version_id	v_00023	Identifies one saved state.
source_path	C:\Business\invoice.pdf	Maps the backup to the original.
timestamp	2026-08-18 17:30	Supports version selection.
size	248 KB	Basic context/integrity information.
sha256	A73F...	Cryptographic fingerprint.
status	VERIFIED_CLEAN	Prevents blind restoration.

8. Step 1 — Set Up Your Environment

The source project uses Python 3.11+ for the desktop agents and lists cryptography, Remberall, FastAPI, SQLite and testing tools in the overall stack. [TRINETRA, pp. 15–16]

For your first build, keep it local. Do not begin with cloud deployment. Build a working recovery engine on dummy data first.

Beginner setup:

- Windows machine.
- Python 3.11+.
- VS Code.
- Git.
- A disposable test directory containing only dummy files.
- A Python virtual environment.

```
mkdir CyberShield
cd CyberShield
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install cryptography pytest
```

Do not execute real ransomware. Use a controlled simulator inside a disposable test directory.

9. Step 2 — Create vaultkeeper/

Your required output is **vaultkeeper/**. The following internal files are a suggested structure; the source does not mandate exact filenames.

```
CyberShield/
|
+-- vaultkeeper/
|   +-- __init__.py
|   +-- manager.py
|   +-- backup.py
|   +-- versioning.py
|   +-- integrity.py
|   +-- encryption.py
|   +-- restore.py
|   +-- recovery.py
|   +-- metadata.py
|   +-- remberall_adapter.py
|   +-- models.py
|
+-- backend/
+-- agent/
+-- ml/
+-- simulator/
+-- tests/
+-- frontend/
+-- docs/
```

10. Step 3 — Build SHA-256 Integrity Checking

Start with hashing because it is simple and fundamental. If you later restore a file, you need a way to answer: **Is this actually the file we expected?**

```
from pathlib import Path
import hashlib

def sha256_file(path: str) -> str:
    h = hashlib.sha256()

    with open(path, "rb") as f:
        while chunk := f.read(1024 * 1024):
            h.update(chunk)

    return h.hexdigest()
```

Understand the code rather than memorizing it: open → read chunks → update the hash → return the final fingerprint.

Test it: hash the same dummy file twice and confirm the hashes match. Change one character and hash it again; the hash should change.

11. Step 4 — Build Basic Backup Creation

Before Remberall or encryption, prove that you can create a clean recovery copy and record its metadata.

```
from pathlib import Path
import shutil
from .integrity import sha256_file

def create_backup(source: str, destination: str):
    source_path = Path(source)
    destination_path = Path(destination)

    destination_path.parent.mkdir(parents=True, exist_ok=True)
    shutil.copy2(source_path, destination_path)

    return {
        "source_path": str(source_path),
        "backup_path": str(destination_path),
        "size": source_path.stat().st_size,
        "sha256": sha256_file(str(source_path)),
    }
```

This is a learning-stage implementation. The final module should connect the same idea to versioning, protected vault storage and the Remberall adapter.

12. Step 5 — Add Versioning

Never overwrite the only recovery copy. Maintain multiple states.

```
report.docx
|
+-- v1
+-- v2
+-- v3
+-- v4
```

The project describes Remberall as providing version snapshots and recovery capabilities. Keep your own version layer thin if Remberall owns that responsibility in the final implementation. [TRINETRA, p. 7]

Rule: a new backup should not destroy the previous known-good recovery point.

13. Step 6 — Understand AES-256

The project specifies AES-256 for backup protection. Encryption and hashing are different:

Mechanism	Purpose
AES-256	Protects backup confidentiality.
SHA-256/checksum	Checks content integrity.
Versioning	Provides historical recovery points.
Protected storage	Reduces the chance that ransomware destroys recovery data.

Use a reputable cryptographic library. Do not invent encryption yourself. The source lists Python's cryptography package and AES-256 in the security stack. [TRINETRA, p. 15]

Build plain backup + hash verification first. Add encryption after the basic recovery path works; this makes debugging much easier.

14. Step 7 — Understand Remberall

The project describes Remberall as an open-source file recovery and versioning system that provides versioning/snapshots, cryptographic integrity verification, deduplication and fast recovery. [TRINETRA, p. 7]

The proposed TRINETRA integration is offline-first, with Remberall integrated into Vaultkeeper and an AES-256 layer added around protected backup storage. [TRINETRA, p. 7]

Important: the supplied document does not specify Remberall's exact function names or API schema. Do not invent those details. Verify the actual repository/API and wrap it behind an adapter.

```
class RemberallAdapter:
    def create_snapshot(self, ...):
        pass

    def list_versions(self, ...):
        pass

    def verify(self, ...):
        pass

    def restore(self, ...):
        pass
```

Your application should depend on the adapter, not on Remberall internals. If the library changes, the adapter is the place you update.

15. Step 8 — Build the Recovery Algorithm

```
ATTACK CONFIRMED
|
v
Receive affected-file list
|
v
For each affected file:
|
+--> Find available versions
+--> Apply trusted time boundary
+--> Choose candidate clean version
+--> Verify candidate integrity
+--> Restore candidate
+--> Verify restored file
+--> Record RECOVERED or FAILED
|
v
Generate recovery summary
```

The official attack flow says Vaultkeeper identifies critical modified files, copies clean versions to the protected vault, verifies integrity, restores safe versions, verifies recovery, and reports status. [TRINETRA, pp. 5–6]

16. Step 9 — Recover One File First

```
customer_data.xlsx

v1 -> 10:00 -> hash A
v2 -> 11:00 -> hash B
v3 -> 12:00 -> hash C
v4 -> 12:06 -> hash D
```

```
Attack -> 12:05
```

```
Choose v3
|
verify stored hash C
|
restore v3
|
hash restored file
|
MATCH -> RECOVERED
```

The attack timestamp/boundary should be supplied by the orchestration layer. Vaultkeeper should not guess it.

17. Step 10 — Recover Many Files

Use per-file results. One unavailable file should not hide the fact that 238 others were recovered.

```
{
  "status": "PARTIAL",
  "total": 250,
  "recovered": 238,
  "failed": 12,
  "files": [
    {"path": "invoice.docx", "status": "RECOVERED"},
    {"path": "accounts.xlsx", "status": "RECOVERED"},
    {"path": "temp.tmp", "status": "FAILED"}
  ]
}
```

This matches the project's requirement to track recovered versus lost files and the dashboard's recovery status model. [TRINETRA, pp. 6, 11]

18. Step 11 — Verify Before and After Restoration

Copying is not recovery. Recovery is only successful when the restored data passes the integrity check required by your design.

```
candidate backup
|
v
verify hash
|
+--> mismatch -> reject
|
v
restore
|
v
verify restored hash
|
+--> mismatch -> recovery failure
|
v
RECOVERED
```


TRINETRA explicitly says a file should not be restored if its hash does not match the original, preventing corrupted or partially encrypted files from being restored. [TRINETRA, pp. 7–8]

19. Step 12 — Use a Recovery State Machine

State	Meaning
IDLE	No recovery job.
RECEIVED	Confirmed recovery request arrived.
ANALYZING	Finding affected files and candidate versions.
VERIFYING	Checking candidate integrity.
RESTORING	Restoring verified clean files.
POST_VERIFY	Checking restored files.
COMPLETE	Everything required was recovered.
PARTIAL	Some files recovered; some failed.
FAILED	Safe recovery could not complete.

These state names are suggested implementation choices. The source requires recovery status/progress but does not prescribe exact state names.

20. Step 13 — Connect to Member 4 / FastAPI

Member 4 owns Policy Engine + FastAPI. The Policy Engine receives the ML risk score, combines ML + Watchdog signals, applies rules, validates the threat, decides Safe/Suspicious/Threat and triggers Vaultkeeper + Enforcer. [TRINETRA, p. 17]

Your module should receive a structured, confirmed recovery event.

```
{
  "event": "RANSOMWARE_CONFIRMED",
  "incident_id": "INC-2026-001",
  "timestamp": "2026-08-18T17:42:10",
  "risk_score": 85,
  "affected_files": [
    "C:/Business/invoice.docx",
    "C:/Business/customers.xlsx"
  ]
}
```

The JSON structure is a suggested interface, not a source-defined API. Agree on the final schema with Member 4 before implementation.

21. Step 14 — Send Progress to the Backend/UI

The broader architecture uses FastAPI, SQLite and WebSocket communication, while the dashboard displays recovery progress and file status. [TRINETRA, pp. 9–12]

Useful recovery events can include:

- recovery_started
- candidate_found

- integrity_verified
- file_restore_started
- file_recovered
- file_failed
- recovery_complete

```
{
  "incident_id": "INC-2026-001",
  "event": "file_recovered",
  "path": "invoice.docx",
  "completed": 17,
  "total": 20
}
```

Event names above are suggested. The source requires the capability, not these exact names.

22. Step 15 — Recovery Report Data

The Business Recovery Report is intended to include the attack timeline, affected/recovered files, downtime, defense actions and system status. Vaultkeeper supplies recovery-specific facts. [TRINETRA, pp. 6, 11–12]

Field	Example
incident_id	INC-2026-001
recovery_started	17:42:11
recovery_completed	17:44:03
files_at_risk	250
files_encrypted	50
files_recovered	238
files_lost	12
integrity_failures	0
recovery_status	PARTIAL

23. Step 16 — Build a Safe Ransomware Simulator

Never use real ransomware for the demo. Use a simulator that operates only inside a disposable test directory and imitates suspicious file modifications.

Demo recipe:

- Create 50–100 dummy files.
- Create several clean versions.
- Modify a subset using the simulator.
- Let Watchdog/ML/Policy Engine confirm the incident.
- Let Enforcer contain the simulator.
- Send the affected-file list to Vaultkeeper.
- Find clean versions and restore them.

- Verify hashes.
- Show recovered/lost counts on the dashboard.

24. Step 17 — Testing Plan

Test	Action	Expected result
Hash stability	Hash the same file twice.	Hashes match.
Hash change	Change one byte/character.	Hash changes.
Backup creation	Back up a dummy file.	Copy + metadata exist.
Versioning	Back up repeatedly.	Previous versions remain.
Clean restore	Restore a trusted version.	Restored hash matches.
Bad candidate	Give a mismatching candidate.	Candidate is rejected.
Selective recovery	Mark 3/10 files affected.	Only 3 need restoration.
Partial recovery	Make one version unavailable.	System reports partial recovery.
Report	Finish recovery.	Summary is generated.
Integration	Send confirmed event.	Vaultkeeper starts and reports progress.

25. Definition of Done

You are done when the following works locally with dummy data:

1. Protected file exists
2. Vaultkeeper creates recovery copy
3. Metadata + SHA-256 are recorded
4. Multiple versions can exist
5. Versions can be listed
6. Confirmed incident identifies affected files
7. Clean candidate is selected
8. Candidate is integrity-verified
9. Candidate is restored
10. Restored file is verified again
11. Result is RECOVERED / FAILED
12. Summary reaches backend
13. Dashboard can show progress

26. Common Beginner Mistakes

Mistake	Better approach
Only one backup	Maintain multiple versions.
Trusting newest backup	Use attack boundary + integrity verification.
Restore without hashes	Verify before and after.
Adding encryption too early	Build copy → hash → version → restore first.
Backing up the entire PC	Start with configured critical folders.
Mixing detection and recovery	Let Policy Engine decide; Vaultkeeper recovers.
Hard-coding Remberall everywhere	Use an adapter.

Mistake	Better approach
Testing with real malware	Use a safe simulator.
Calling a copied file 'recovered'	Require post-restore verification.

27. How You Connect With Every Team Member

Member	Role	Your connection
1	Windows Agent / Watchdog	Affected-file/activity context.
2	Detection + ML	Risk/detection context through orchestration.
4	Backend + Policy Engine	Main integration: confirmed events, status, results.
5	Frontend	Progress, counts, per-file results, report data.
6	Integration + QA	Simulator and end-to-end recovery testing.

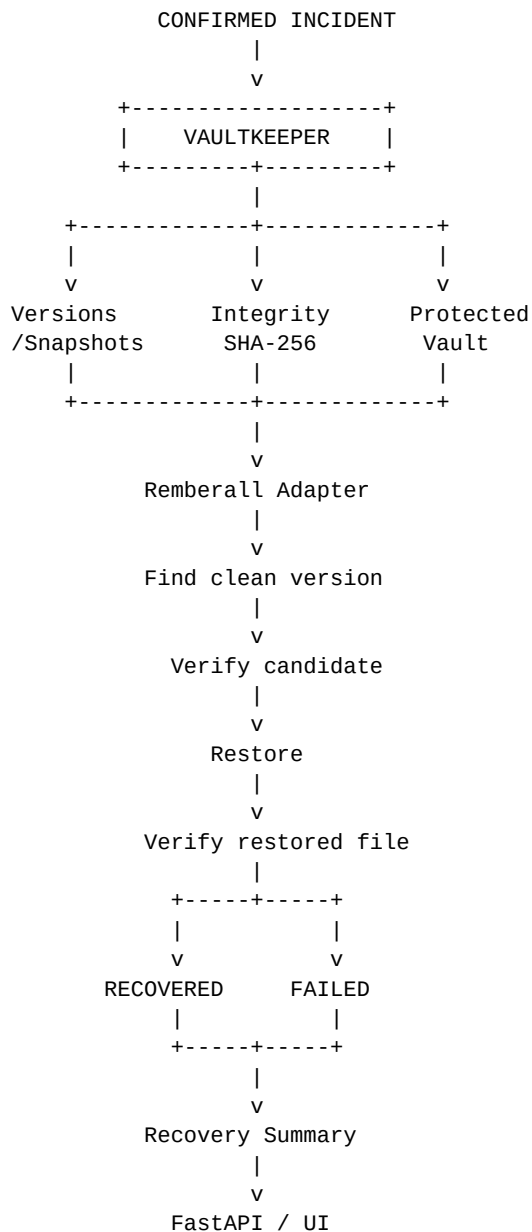
Documented team flow: **M1 detects** → **M2 analyzes** → **M4 decides** → **M6 defends + M3 recovers** → **M5 visualizes**. [TRINETRA, p. 17]

28. Exact Learning/Build Order

1. Learn backup, version, hash, integrity and restore concepts.
2. Create vaultkeeper/ and a disposable test_data/ directory.
3. Implement SHA-256.
4. Write hashing tests.
5. Implement basic backup copying.
6. Store metadata.
7. Implement version listing.
8. Implement one-file restore.
9. Verify the restored hash.
10. Implement multi-file recovery.
11. Track recovered/failed files.
12. Add recovery-job status.
13. Create the Remberall adapter interface.
14. Inspect the actual Remberall repository/API before writing integration code.
15. Integrate snapshots/versioning/recovery through the adapter.
16. Add AES-256 protected backup storage.
17. Connect FastAPI.
18. Connect real-time progress.
19. Run the safe simulator.
20. Run end-to-end tests.

21. Document the module and prepare the judge demo.

29. Final Architecture



The source architecture describes the desktop agent, local FastAPI server, SQLite database, WebSocket updates and recovery files in the vault. [TRINETRA, pp. 9–10]

30. Judge Questions and Answers

Question	Answer
What is your role?	I own Vaultkeeper, the backup and recovery module.
What happens after ransomware is confirmed?	Vaultkeeper receives the recovery event, identifies affected files, finds clean versions, verifies them, restores them and verifies them again.
Why not restore the newest backup?	It may have been created after the attack or may contain compromised data.
Why SHA-256?	It provides a cryptographic fingerprint for integrity checking.

Question	Answer
Why versioning?	It provides multiple historical recovery points.
Why Remberall?	The project uses it for recovery/versioning, integrity verification, deduplication and fast/selective recovery.
Why AES-256?	It protects the confidentiality of backup data as specified by the project.
What if a file cannot be recovered?	Mark it failed/lost and report it accurately.
How do you test?	Use a controlled simulator and disposable dummy files, never real ransomware.

31. Your 30-Second Explanation

"I am the Recovery Engineer for TRINETRA, responsible for the Vaultkeeper module. My job starts when the Policy Engine confirms a ransomware incident. Vaultkeeper maintains protected, versioned recovery copies and their integrity information. During an attack, it identifies affected files, finds the latest trusted clean versions, verifies their cryptographic integrity, restores only the damaged files, verifies them again, and sends recovered-versus-lost information to the backend and dashboard. Remberall is integrated as the project's recovery and versioning layer, while AES-256 protects the backup data."

32. Final Mental Model

BACKUP

-> Keep clean copies

VERSION

-> Keep multiple safe points

HASH

-> Know whether a copy is trustworthy

VAULT

-> Protect recovery copies

REMBERALL

-> Project's recovery/versioning layer

RECOVER

-> Put clean files back

VERIFY

-> Never claim success without integrity verification

REPORT

-> Tell the system what was recovered/lost

Your module is the safety net of TRINETRA. Detection tells the team that an attack is happening. Enforcer stops it. Your job is to make sure the user's business data can come back safely.

33. Source Notes and Boundaries

Primary source: TRINETRA — Bharat's Next-Generation Cyber Resilience Platform, Version 2.0, dated August 16, 2026. Key areas used: core agents/Vaultkeeper (pp. 4–8), architecture (pp. 9–10), dashboard/recovery reporting (pp. 11–13), technical stack (pp. 15–16), team organization (p. 17), conclusion (p. 20).

Secondary source: Smart India Hackathon 2026 deck for Cipher Syndicate. Key areas used: proposed solution/uniqueness (p. 2), technical approach (p. 3), feasibility/risks (p. 4), impact/recovery flow (p. 5), research insights (p. 6).

Boundary: The supplied documents specify the architecture and intended capabilities but do not define every Remberall API, exact database schema, exact FastAPI endpoint, exact key-management mechanism, or exact file names. Those parts of this guide are suggested implementation choices. Verify the actual Remberall repository/API before coding the adapter.